

MIP_SQLITE_PHYSICAL_DATA_MODEL.md

Mainframe Intelligence Platform (MIP)

SQLite Physical Data Model

Version: 1.0

Status: Implementation Ready

Classification: Engineering Specification

Owner: MIP Engineering Team

Executive Summary

SQLite is the persistence layer for MIP v1.

Purpose:

- Store repository metadata
- Store discovered relationships
- Store graph nodes
- Store graph edges
- Store lineage paths
- Store impact analysis results
- Store modernization recommendations

Database File:

sqlite/mip.db

Database Design Principles

Metadata First

Store normalized metadata.

Graph Friendly

Enable efficient graph construction.

Explainable

Store evidence for every relationship.

Scalable

Support large enterprise repositories.

Target:

100,000+ Programs

20,000+ Jobs

500,000+ Tables

10M+ Relationships

Database Architecture

Repository

↓

Metadata Tables

↓

Relationship Tables

↓

Graph Tables

↓

Lineage Tables

↓

Impact Tables

↓

Recommendation Tables

Core Entity Model

Application

↓

Domain

↓

Capability

↓

Program

↓

Relationships

↓

Graph

↓

Lineage

↓

Impact

↓

Recommendations

Table: application

```
CREATE TABLE application (  
    application_id TEXT PRIMARY KEY,  
    application_name TEXT NOT NULL,  
    description TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Purpose:

Represents enterprise applications.

Examples:

```
TSYS  
  
Core Banking  
  
Billing
```

Table: domain

```
CREATE TABLE domain (  
    domain_id TEXT PRIMARY KEY,  
    application_id TEXT,  
    domain_name TEXT,  
    description TEXT,  
    FOREIGN KEY(application_id)  
        REFERENCES application(application_id)  
);
```

Examples:

Authorization

Settlement

Customer Management

Table: capability

```
CREATE TABLE capability (  
    capability_id TEXT PRIMARY KEY,  
    domain_id TEXT,  
    capability_name TEXT,  
    description TEXT,  
    FOREIGN KEY(domain_id)  
        REFERENCES domain(domain_id)  
);
```

Examples:

Card Activation

Balance Inquiry

Payment Processing

Table: program

```
CREATE TABLE program (  
    program_id TEXT PRIMARY KEY,  
    program_name TEXT,  
    language TEXT,  
    file_path TEXT,
```

```
lines_of_code INTEGER,  
  
complexity_score REAL,  
  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Table: copybook

```
CREATE TABLE copybook (  
  
    copybook_id TEXT PRIMARY KEY,  
  
    copybook_name TEXT,  
  
    file_path TEXT,  
  
    field_count INTEGER  
);
```

Table: job

```
CREATE TABLE job (  
  
    job_id TEXT PRIMARY KEY,  
  
    job_name TEXT,  
  
    step_count INTEGER,  
  
    scheduler TEXT  
);
```

Table: job_step

```
CREATE TABLE job_step (  
  
    step_id TEXT PRIMARY KEY,  
  
    job_id TEXT,
```

```
    step_name TEXT,  
  
    program_name TEXT,  
  
    FOREIGN KEY(job_id)  
        REFERENCES job(job_id)  
);
```

Table: dataset

```
CREATE TABLE dataset (  
  
    dataset_id TEXT PRIMARY KEY,  
  
    dataset_name TEXT,  
  
    dataset_type TEXT  
);
```

Examples:

```
VSAM  
  
SEQ  
  
GDG  
  
FLATFILE
```

Table: table_metadata

```
CREATE TABLE table_metadata (  
  
    table_id TEXT PRIMARY KEY,  
  
    schema_name TEXT,  
  
    table_name TEXT  
);
```

Table: column_metadata

```
CREATE TABLE column_metadata (  
    column_id TEXT PRIMARY KEY,  
    table_id TEXT,  
    column_name TEXT,  
    data_type TEXT,  
    FOREIGN KEY(table_id)  
        REFERENCES table_metadata(table_id)  
);
```

Relationship Repository

This is the most important table.

Table: relationship

```
CREATE TABLE relationship (  
    relationship_id TEXT PRIMARY KEY,  
    source_id TEXT,  
    target_id TEXT,  
    relationship_type TEXT,  
    confidence REAL,  
    evidence TEXT,  
    source_file TEXT,  
    line_number INTEGER  
);
```

Examples:

CALLS

READS

WRITES

EXECUTES

USES

PRODUCES

CONSUMES

Graph Repository

Table: graph_node

```
CREATE TABLE graph_node (  
    node_id TEXT PRIMARY KEY,  
    node_type TEXT,  
    node_name TEXT,  
    metadata_json TEXT  
);
```

Table: graph_edge

```
CREATE TABLE graph_edge (  
    edge_id TEXT PRIMARY KEY,  
    source_node TEXT,  
    target_node TEXT,  
    edge_type TEXT,
```

```
        confidence REAL
    );
```

Lineage Repository

Table: lineage_path

```
CREATE TABLE lineage_path (
    path_id TEXT PRIMARY KEY,
    source_node TEXT,
    target_node TEXT,
    path_type TEXT,
    hop_count INTEGER,
    confidence_score REAL,
    created_at TIMESTAMP
);
```

Table: lineage_step

```
CREATE TABLE lineage_step (
    step_id TEXT PRIMARY KEY,
    path_id TEXT,
    sequence_number INTEGER,
    node_id TEXT,
    FOREIGN KEY(path_id)
        REFERENCES lineage_path(path_id)
);
```

Impact Repository

Table: impact_analysis

```
CREATE TABLE impact_analysis (  
    impact_id TEXT PRIMARY KEY,  
    entity_id TEXT,  
    impact_score REAL,  
    severity TEXT,  
    blast_radius INTEGER,  
    analysis_date TIMESTAMP  
);
```

Table: impact_asset

```
CREATE TABLE impact_asset (  
    impact_asset_id TEXT PRIMARY KEY,  
    impact_id TEXT,  
    affected_entity_id TEXT,  
    entity_type TEXT,  
    FOREIGN KEY(impact_id)  
        REFERENCES impact_analysis(impact_id)  
);
```

Recommendation Repository

Table: recommendation

```
CREATE TABLE recommendation (  
    recommendation_id TEXT PRIMARY KEY,  
    recommendation_type TEXT,  
    entity_id TEXT,  
    confidence REAL,  
    recommendation_text TEXT,  
    created_at TIMESTAMP  
);
```

Recommendation Types

```
SERVICE_CANDIDATE  
API_CANDIDATE  
EVENT_CANDIDATE  
WAVE_1  
WAVE_2  
WAVE_3  
WAVE_4  
HIGH_RISK  
DEAD_CODE
```

Agent Execution Repository

Table: agent_execution

```
CREATE TABLE agent_execution (  
    execution_id TEXT PRIMARY KEY,  
    agent_name TEXT,  
    start_time TIMESTAMP,  
    end_time TIMESTAMP,  
    status TEXT,  
    confidence REAL  
);
```

Evidence Repository

Table: evidence

```
CREATE TABLE evidence (  
    evidence_id TEXT PRIMARY KEY,  
    entity_id TEXT,  
    source_file TEXT,  
    line_number INTEGER,  
    evidence_text TEXT  
);
```

Index Strategy

Programs

```
CREATE INDEX idx_program_name  
ON program(program_name);
```

Relationships

```
CREATE INDEX idx_relationship_source  
ON relationship(source_id);
```

```
CREATE INDEX idx_relationship_target  
ON relationship(target_id);
```

```
CREATE INDEX idx_relationship_type  
ON relationship(relationship_type);
```

Graph

```
CREATE INDEX idx_graph_source  
ON graph_edge(source_node);
```

```
CREATE INDEX idx_graph_target  
ON graph_edge(target_node);
```

Impact

```
CREATE INDEX idx_impact_score  
ON impact_analysis(impact_score);
```

Query Examples

Programs Calling TS2AUTH

```
SELECT *  
FROM relationship
```

```
WHERE target_id='TS2AUTH'  
AND relationship_type='CALLS';
```

Programs Updating CARD_MASTER

```
SELECT *  
FROM relationship  
WHERE target_id='CARD_MASTER'  
AND relationship_type='WRITES';
```

Critical Components

```
SELECT *  
FROM impact_analysis  
WHERE severity='CRITICAL';
```

Backup Strategy

Daily:

```
mip.db  
  
↓  
  
mip_backup.db
```

Future:

```
SQLite  
  
↓  
  
PostgreSQL
```

Migration path must remain compatible.

Phase 1 Scope

Implement:

- ✓ Metadata Tables
 - ✓ Relationship Tables
 - ✓ Graph Tables
 - ✓ Lineage Tables
 - ✓ Impact Tables
 - ✓ Recommendation Tables
 - ✓ Agent Execution Tables
-

Phase 2 Scope

Add:

- ✓ Versioning
 - ✓ Change Tracking
 - ✓ Audit History
 - ✓ Incremental Scanning
-

Success Criteria

Database supports:

- 100K+ Programs
- 10M+ Relationships
- Fast Graph Construction
- Fast Impact Analysis
- Fast Lineage Queries

on a developer laptop.

MIP Principle

SQLite stores facts.

NetworkX stores relationships.

Lineage stores movement.

Impact stores consequences.

Reasoning stores intelligence.

Modernization stores strategy.

End of Document